EJS (Embedded JavaScript)

เป็นเครื่องมือ template engine ที่ได้รับความนิยมในการพัฒนาเว็บแอปพลิเคชันด้วย Node.js ซึ่งช่วยในการสร้างหน้า HTML ที่สามารถฝังโค้ด JavaScript เพื่อสร้างเนื้อหาที่เปลี่ยนแปลงได้ (dynamic content) โดยมีความสามารถและฟีเจอร์ที่น่าสนใจ ดังนี้

ความสามารถของ EJS

1. การฝังโค้ด JavaScript ใน HTML

EJS ช่วยให้คุณสามารถเขียนโค้ด JavaScript ภายในไฟล์ HTML ได้โดยการใช้ syntax แบบ <%= %> หรือ <% %> เพื่อประมวลผลข้อมูลหรือเงื่อนไขต่างๆ

- <%= %>: ใช้สำหรับแสดงผลค่า (escape HTML)
- <%- %>: ใช้สำหรับแสดงผลค่าโดยไม่ทำการ escape (raw HTML)
- <% %>: ใช้สำหรับควบคุมโค้ด (ไม่แสดงผลออกมา)

ตัวอย่าง

```
<h1><%= title %></h1> <!-- แสดงค่าตัวแปร title -->
<% if (user) { %>
  <p>Welcome, <%= user.name %>!</p>
<% } else { %>
  <p>Please log in.</p>
<% } %>
```

2. การใช้ตัวแปรจากเซิร์ฟเวอร์

EJS สามารถรับข้อมูลจากเซิร์ฟเวอร์และแสดงผลลงใน HTML ได้อย่างง่ายดาย เช่น คุณสามารถส่งตัวแปรหรือข้อมูลจาก route ของ Express.js ไปยังไฟล์ EJS เพื่อแสดงผล

ตัวอย่าง

```
app.get('/', (req, res) => {
  res.render('index', { title: 'Home Page', user: { name: 'John' } });
});
```

3. การวนลูปและเงื่อนไข (Looping and Conditionals)

EJS สนับสนุนการใช้โค้ด JavaScript เพื่อจัดการลูปและเงื่อนไขภายในเทมเพลต

ตัวอย่าง

```
<ul>
<% items.forEach(function(item) { %>
  <li><%= item %></li>
<% }) %>
</ul>
```

4. การใช้ฟังก์ชันและการคำนวณในเทมเพลต

คุณสามารถใช้ฟังก์ชัน JavaScript หรือการคำนวณต่างๆ ภายในเทมเพลต EJS ได้

ตัวอย่าง

```
<p>Random number: <%= Math.random() %></p>
```

5. การแบ่งไฟล์เทมเพลตย่อย (Partials)

EJS สนับสนุนการรวมไฟล์เทมเพลตย่อย (partials) ซึ่งช่วยให้การจัดการโครงสร้างของหน้าเว็บทำได้ง่ายขึ้น โดยสามารถสร้างไฟล์ย่อยๆ สำหรับส่วนต่างๆ ของหน้าเว็บ เช่น header, footer หรือ sidebar แล้วนำมารวมเข้ากับหน้าหลัก

ตัวอย่าง

```
<%- include('header') %>
<h1>Main Content</h1>
<%- include('footer') %>
```

6. HTML Escaping

EJS จะทำการแปลงค่าไปเป็นตัวอักษรพิเศษที่มีความหมายเฉพาะในภาษา HTML ให้เป็นรูปแบบที่เบราว์เซอร์เข้าใจและแสดงผลได้อย่างถูกต้องโดยอัตโนมัติเมื่อใช้ <%= %> เพื่อป้องกัน XSS (Cross-Site Scripting) และปัญหาด้านความปลอดภัยต่างๆ ที่อาจเกิดจากการฝังข้อมูลจากผู้ใช้โดยตรงลงใน HTML

ตัวอย่าง

```
<p><%= userInput %></p>
```

7. การจัดการการจัดรูปแบบ (Layout Management)

แม้ว่า EJS จะไม่รองรับฟีเจอร์ layout โดยตรงเหมือนบาง template engine เช่น Pug หรือ Handlebars แต่คุณยังสามารถใช้ partials และ includes เพื่อแบ่งส่วนเทมเพลตได้ ทำให้การจัดการโครงสร้างของ layout ยังคงทำได้อย่างยืดหยุ่น

ตัวอย่าง

คุณสามารถรวม header.ejs และ footer.ejs ในทุกๆ หน้าเพื่อให้โครงสร้างของหน้าเว็บคงที่

8. สนับสนุนการทำงานร่วมกับ Express.js

EJS เป็นหนึ่งใน template engine ที่สามารถทำงานร่วมกับ Express.js ได้อย่างง่ายดาย โดยสามารถตั้งค่าให้ใช้ EJS ใน Express.js ได้ด้วยการกำหนด view engine เป็น ejs

ตัวอย่างการตั้งค่าใน Express

```
const express = require('express');
const app = express();

app.set('view engine', 'ejs'); // ตั้งค่า EJS เป็น view engine
```

9. การจัดการข้อมูลแบบไดนามิก

EJS รองรับการแสดงผลข้อมูลที่เปลี่ยนแปลงได้แบบไดนามิก เช่น การแสดงข้อมูลจากฐานข้อมูลหรือ API โดยข้อมูลเหล่านี้สามารถถูกประมวลผลและแสดงผลใน HTML ได้อย่างง่ายดาย

ตัวอย่าง

```
app.get('/users', (req, res) => {
  const users = [
    { name: 'John', age: 25 },
    { name: 'Jane', age: 30 },
  ];
  res.render('users', { users });
});
```

ไฟล์ users.ejs

```
<ul>
  <% users.forEach(function(user) { %>
    <li><%= user.name %> - <%= user.age %> years old</li>
  <% }) %>
</ul>
```

10. รองรับการใช้งานทั้งฝั่งเซิร์ฟเวอร์และไคลเอนต์

แม้ว่าการใช้งาน EJS ส่วนใหญ่จะอยู่ฝั่งเซิร์ฟเวอร์ แต่ EJS ก็สามารถนำไปใช้ฝั่งไคลเอนต์ได้เช่นกัน โดยการรวมไฟล์ EJS เข้ากับ HTML หรือ JavaScript ที่รันบนเบราว์เซอร์ได้

ตัวอย่าง (ฝั่งไคลเอนต์)

```
<script src="https://cdn.jsdelivr.net/npm/ejs/ejs.min.js"></script>
<script>
  const template = '<h1>Hello, <%= name %>!</h1>';
  const html = ejs.render(template, { name: 'John' });
  document.body.innerHTML = html;
</script>
```

EJS เป็นเครื่องมือที่มีประสิทธิภาพและง่ายต่อการใช้งานในการสร้างเว็บแอปพลิเคชันแบบไดนามิก ซึ่งมาพร้อมกับฟีเจอร์ที่ช่วยให้นักพัฒนาสามารถฝังโค้ด JavaScript ลงใน HTML ได้ง่ายๆ เช่น การวนลูป การเงื่อนไข การจัดการเทมเพลตย่อย (partials) และสามารถทำงานร่วมกับ Express.js ได้อย่างไร้รอยต่อ

ในกรณีที่มีหลายหน้าในแอปพลิเคชันที่พัฒนาด้วย Node.js และใช้ EJS เป็น Template Engine มีหลายเทคนิคในการส่งข้อมูลระหว่างหน้าต่างๆ ขึ้นอยู่กับลักษณะของข้อมูลที่ต้องการส่งผ่านและวิธีการที่คุณต้องการใช้งาน สำหรับ Express.js จะมีวิธีหลักๆ ในการส่งข้อมูลระหว่างหน้าได้ดังนี้

1. การส่งข้อมูลโดยตรงผ่าน res.render()

คุณสามารถส่งข้อมูลไปยังหน้า EJS แต่ละหน้าโดยตรงจาก res.render() เมื่อทำการส่ง response จาก route ใน Express.js ข้อมูลจะถูกส่งผ่านไปในรูปแบบของอ็อบเจ็กต์ และสามารถใช้ในเทมเพลต EJS ได้ทันที

ตัวอย่าง

```
app.get('/page1', (req, res) => {
  res.render('page1', { message: 'Welcome to Page 1!' });
});

app.get('/page2', (req, res) => {
  res.render('page2', { message: 'This is Page 2', data: [1, 2, 3] });
});
```

ในหน้า page1.ejs

```
<h1><%= message %></h1>
```

ในหน้า page2.ejs

```
<h1><%= message %></h1>
```

```
<ul>
  <% data.forEach(function(item) { %>
    <li><%= item %></li>
  <% }) %>
</ul>
```

2. การใช้ URL Parameters

การส่งข้อมูลระหว่างหน้าผ่านทาง URL Parameters คือการแนบข้อมูลไปกับ URL เมื่อทำการนำทางไปยังหน้าต่างๆ โดยใช้การส่งค่าผ่าน query string หรือ route parameters ซึ่งเป็นวิธีการส่งข้อมูลผ่าน URL ที่ได้รับความนิยม

ตัวอย่าง

```
// Route ที่มี parameter ใน URL
app.get('/page/:id', (req, res) => {
  const id = req.params.id;
  res.render('page', { id });
});
```

ในหน้า page.ejs

```
<h1>Page ID: <%= id %></h1>
```

- การเข้าถึง URL เช่น /page/5 จะส่งค่าของ id เป็น 5 ไปยังหน้าดังกล่าว

หรือสามารถใช้ query string ได้เช่นกัน

```
app.get('/page', (req, res) => {
  const name = req.query.name;
  res.render('page', { name });
});
```

ในหน้า page.ejs

<h1>Hello, <%= name %>!</h1>

- การเข้าถึง URL เช่น /page?name=John จะส่งค่าของ name เป็น John

ใน Express.js โครงสร้าง URL ที่มีเครื่องหมาย : (colon) นำหน้า เช่น :id เรียกว่า **Route Parameters** หรือ **Path Parameters** ซึ่งใช้เพื่อจับค่าที่ปรากฏใน URL และทำให้เราสามารถดึงข้อมูลที่อยู่ในเส้นทาง (path) ของ URL ได้

การทำงานของ Route Parameters

- **:id** ในเส้นทาง '/page/:id' คือค่าที่เปลี่ยนแปลงได้ซึ่งผู้ใช้ส่งมาใน URL โดย id คือชื่อของพารามิเตอร์ที่เราต้องการดึงออกมาใช้ในฟังก์ชัน
- เมื่อมีการเรียก URL เช่น /page/123 ตัวเลข 123 จะถูกจับและเก็บไว้ในตัวแปร id ซึ่งเราสามารถเข้าถึงได้ผ่าน req.params.id

ตัวอย่าง

```
const express = require('express');
const app = express();


app.get('/page/:id', (req, res) => {
    const pageId = req.params.id;  // ดึงค่าจาก :id ใน URL
    res.send(`You are viewing page with ID: ${pageId}`);
});
app.listen(3000, () => {
    console.log('Server is running on http://localhost:3000');
});
```

ในตัวอย่างนี้

- เมื่อมีการเรียก URL เช่น http://localhost:3000/page/123, ค่าที่ตามหลัง :id คือ 123 จะถูกจับและเก็บไว้ใน req.params.id
- จากนั้นโค้ดจะส่งข้อความกลับไปที่ client ว่า You are viewing page with ID: 123

การใช้งานหลาย Route Parameters

คุณสามารถมีหลาย Route Parameters ใน URL ได้โดยการเพิ่ม :parameterName ต่อกัน เช่น

```
app.get('/user/:userId/book/:bookId', (req, res) => {
    const userId = req.params.userId;
    const bookId = req.params.bookId;
    res.send(`User ID: ${userId}, Book ID: ${bookId}`);
});
```

ในตัวอย่างนี้:

- เมื่อมีการเรียก URL เช่น http://localhost:3000/user/5/book/10
- userId จะเท่ากับ 5
- bookId จะเท่ากับ 10

การใช้งาน Route Parameters ช่วยให้การสร้าง URL ที่มีโครงสร้างสื่อความหมายและการจัดการข้อมูลในเส้นทาง URL ง่ายและยืดหยุ่นมากขึ้น

3. การใช้ Sessions

Session เป็นอีกหนึ่งวิธีที่นิยมใช้ในการส่งข้อมูลระหว่างหลายหน้าในแอปพลิเคชัน ข้อมูลจะถูกเก็บไว้ในเซิร์ฟเวอร์ (หรือในคุกกี้แบบเข้ารหัส) และสามารถนำมาใช้ใน request ถัดไปได้ ทำให้การส่งข้อมูลระหว่างหลายหน้าเป็นไปได้อย่างราบรื่น

ติดตั้งไลบรารี express-session

```
npm install express-session
```

การตั้งค่า Session ใน Express.js

```
const session = require('express-session');

app.use(session({
  secret: 'mySecretKey',
  resave: false,
  saveUninitialized: true
}));

app.get('/set-session', (req, res) => {
  req.session.user = { name: 'John', age: 30 };
  res.send('Session has been set');
});

app.get('/get-session', (req, res) => {
  const user = req.session.user;
  if (user) {
    res.render('user', { user });
  } else {
    res.send('No session found');
```

```
  }
});
```

ในหน้า user.ejs

```
<h1>User Info</h1>
<p>Name: <%= user.name %></p>
<p>Age: <%= user.age %></p>
```

- เมื่อเข้าถึง /set-session จะทำการบันทึกข้อมูล session และเมื่อเข้าถึง /get-session จะดึงข้อมูล session ขึ้นมาแสดงผล

4. การใช้ Cookies

Cookies ใช้สำหรับการเก็บข้อมูลฝั่งไคลเอนต์ โดยสามารถเก็บค่าบางอย่างเช่น user preferences หรือ session ID ซึ่งสามารถใช้สำหรับการส่งข้อมูลระหว่างหลายหน้าได้

ติดตั้งไลบรารี cookie-parser

```
npm install cookie-parser
```

การตั้งค่า Cookies ใน Express.js

```
const cookieParser = require('cookie-parser');
app.use(cookieParser());

app.get('/set-cookie', (req, res) => {
  res.cookie('user', { name: 'John', age: 30 });
  res.send('Cookie has been set');
});

app.get('/get-cookie', (req, res) => {
```

```
  const user = req.cookies.user;
  res.render('user', { user });
});
```

ในหน้า user.ejs

```
<h1>User Info from Cookies</h1>
<p>Name: <%= user.name %></p>
<p>Age: <%= user.age %></p>
```

- เมื่อเข้าถึง /set-cookie จะทำการตั้งค่า cookie และเมื่อเข้าถึง /get-cookie จะดึงค่าจาก cookie มาแสดงผล

5. การใช้ Hidden Inputs ในฟอร์ม

หากคุณต้องการส่งข้อมูลระหว่างหลายหน้าโดยใช้ฟอร์ม คุณสามารถใช้ hidden input เพื่อเก็บข้อมูลที่ต้องการส่งผ่านฟอร์มโดยไม่ให้ผู้ใช้เห็นได้

ตัวอย่าง

```
<form action="/submit" method="post">
  <input type="hidden" name="userId" value="<%= user.id %>">
  <button type="submit">Submit</button>
</form>
```

ใน Express.js

```
app.post('/submit', (req, res) => {
  const userId = req.body.userId;
  // ใช้ข้อมูล userId ทำบางอย่าง
  res.send(User ID: ${userId});
```

});

6. การใช้ Frontend Storage (LocalStorage/SessionStorage)

ถ้าต้องการเก็บข้อมูลฝั่งไคลเอนต์สามารถใช้ Web Storage API เช่น localStorage หรือ sessionStorage ซึ่งจะเก็บข้อมูลในเบราว์เซอร์ และดึงข้อมูลนี้ขึ้นมาใช้งานได้ในหน้าถัดไป

ตัวอย่าง

```
<script>
  // เก็บข้อมูลลงใน localStorage
  localStorage.setItem('username', 'John Doe');

  // ดึงข้อมูลจาก localStorage
  const username = localStorage.getItem('username');
  document.getElementById('user').textContent = username;
</script>
<p id="user"></p>
```

การส่งข้อมูลระหว่างหลายหน้าใน Node.js และ EJS มีหลากหลายเทคนิคที่สามารถเลือกใช้ตามความเหมาะสมกับกรณีต่างๆ

- ส่งข้อมูลโดยตรงผ่าน res.render()
- ใช้ URL Parameters หรือ Query Strings
- ใช้ Sessions เพื่อเก็บข้อมูลฝั่งเซิร์ฟเวอร์
- ใช้ Cookies เพื่อเก็บข้อมูลฝั่งไคลเอนต์
- ใช้ hidden inputs ในฟอร์ม
- ใช้ Web Storage (localStorage หรือ sessionStorage)